# Xi'an Jiaotong-Liverpool University

# 西交利物浦大學

| PAPER CODE | EXAMINER | DEPARTMENT | TEL |
|---|---|---|---|
| CPT304 | | CPT | |

## 2nd SEMESTER 2023/24 Final EXAMINATION

## BACHELOR DEGREE – Year 4

## Software Engineering II

## TIME ALLOWED: 2 Hours

---

## INSTRUCTIONS TO CANDIDATES

1、 **This is a closed-book examination, which is to be written without books or notes.**

2、 **Total marks available are 100. This will count for 80% in the final marks.**

3、 **This exam consists of NINE (9) questions printed on 4 pages of papers including the cover page.**

4、 **Answer all questions. There is NO penalty for providing a wrong answer.**

5、 **Answers should be written in the answer booklet(s) provided.**

6、 **Only English solutions are accepted.**

7、 **All materials must be returned to the exam invigilator upon completion of the exam. Failure to do so will be deemed academic misconduct and will be dealt with accordingly.**

XJTLU Academic Use Only by 2257453
Use Responsibly & Respect Copyright

XJTLU Academic Use Only by 2257453
Use Responsibly & Respect Copyright

**Answer all questions. Write your answers in the provided answer booklet.**

**Question 1**

In "No Silver Bullet," Frederick P. Brooks discusses the concept of essential difficulties in software engineering, highlighting complexity, conformity, changeability, and invisibility as key challenges.

1. Briefly explain each of these essential difficulties. **(8 marks)**

2. Evaluate the significance of these essential difficulties and their relevance to contemporary software engineering practices, methodologies, and technologies. **(10 marks)**

**Question 2**

In the context of software engineering, cohesion and coupling are two fundamental principles that govern the design and organization of classes, modules, and subsystems.

1. Define the terms "cohesion" and "coupling" within the context of software engineering. **(6 marks)**

2. Explain why these concepts are significant when designing and developing software. **(4 marks)**

**Question 3**

Design patterns are widely used in the field of software engineering to solve commonly occurring problems in software design. Understanding these patterns will equip you with the knowledge to design robust and scalable systems.

1. Define the following concepts in the context of software design. **(6 marks)**

   - Creational Design Patterns
   - Structural Design Patterns
   - Behavioral Design Patterns

2. The strategy design pattern is a behavioral pattern that allows the strategy (or algorithm) to be selected at runtime. Explain in detail the key components of the Strategy design pattern, using a real-world example to illustrate your explanation. **(5 marks)**

3. Draw a UML diagram for the Strategy patterns. **(6 marks)**

XJTLU Academic Use Only by 2257453
Use Responsibly & Respect Copyright

**Question 4**

Software Product Lines (SPL) is a software engineering methodology that uses platforms and reusable components to create a series of related applications.

1. Explain the concept of Software Product Lines **(2 marks)**

2. Discuss THREE of its advantages. **(6 marks)**

3. Let's think about a web development company that developed Software Product Lines for e-commerce applications.

   The company has decided to create two version of products as follows:

   - Product 1 includes basic e-commerce functionalities like a simple checkout system and basic inventory management.
   - Product 2 includes advanced features like sophisticated inventory and supply chain management, integration with multiple payment gateways, and advanced analytics.

   The company understands that its clients use different web hosting environments like Apache, NGINX, or cloud services like AWS or Azure. It wants its products to work smoothly in these different environments. So, it further specializes each product to ensure compatibility and optimize performance for each of these hosting environments.

   What are the specializations the company has used when extending the product lines? Justify your answers. **(6 marks)**

**Question 5**

Component-Based Software Engineering (CBSE) is a branch of software engineering that emphasizes the design and construction of software systems using reusable software components.

1. Elaborate THREE main characteristics of a component in the context of CBSE. **(6 marks)**

2. Briefly explain the component identification sub-process in the context of development with reuse for CBSE. **(6 marks)**

XJTLU Academic Use Only by 2257453
Use Responsibly & Respect Copyright

**Question 6**
Define meritocracy in the context of open-source development. **(3 marks)**

**Question 7**
Explain the role of user feedback in open-source projects. **(3 marks)**

**Question 8**
How does transparency factor into open-source development? **(3 marks)**

**Question 9**
Consider a function *f(x1, x2)* where the values of *x1* and *x2* are defined to be

$$a <= x1 <= b \quad \text{and} \quad c <= x2 <= d$$

Assume the equivalence classes for *x1* is *{a, a+1, ..., ta}, {ta+1, ..., tb}, {tb+1, tb+2, ..., b}* and the equivalence classes for *x2* is *{c, c+1, ..., tc}, {tc+1, tc+2, ..., td}, {td+1, td+2, ..., d}*.

1. In weak normal equivalence class testing, what is the minimum number of test cases? **(1 mark)**

2. Using graphical representation, show the minimum test cases one could generate using weak normal equivalence class testing. **(3 marks)**

3. In strong normal equivalence class testing, how many test cases are generated? **(1 mark)**

4. Explain in detail why a weak normal equivalence class testing generates less test cases compare to a strong normal equivalence class testing? **(6 marks)**

5. List the equivalence classes of *x1* and *x2* for robustness testing. **(2 marks)**

6. Using graphical representation, show the minimum test cases you could generate using weak robust equivalence class testing. **(4 marks)**

7. Using graphical representation, show the test cases you could generate using strong robust equivalence class testing. **(3 marks)**

**The end of the paper**